

Phase - 3

Execution context and call stack

Execution context ✓

→ Environment where JavaScript code is executed.

→ Whenever JS code runs it creates
— an execution context ✓ —

⇒ There are mainly 3 types

① Global execution context ✓

→ Created when JS file starts

→ created once

② Function Execution context ✓

→ created every time a function is
called ✓

contains that function variables, parameters etc. —

① Eval Execution context

var x = 10
Eval ("var y = 20; console.log(x+y);")

→ created when eval() executes code

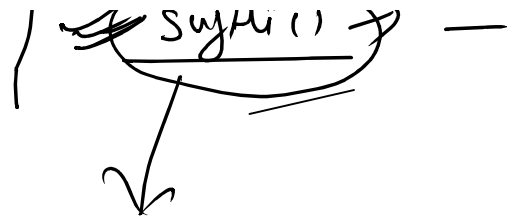
→ Rarely used in modern JS.

Parts or components of execution context

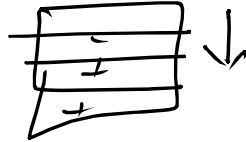
②

memory components	code components
Stores your variables and functions as <u>key-value</u> - pairs x: → <u>undefined</u> → <u>10</u> <u>sayHi</u>: <u>function()</u>	code runs and executes your code line-by-line → console.log(x) → <u>undefined</u> x = 10 <u>sayHi()</u>

0

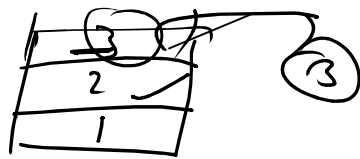


call stack →



- a stack that keeps track of which function is currently running

LIFO → Last in, first out



X

```
console.log("Hello world");  
const a = 10  
function sayHi() {  
  return ("Hi")  
}  
3  
console.log(a)  
sayHi()
```

The diagram illustrates the Global execution context, divided into two main sections: memory and code.

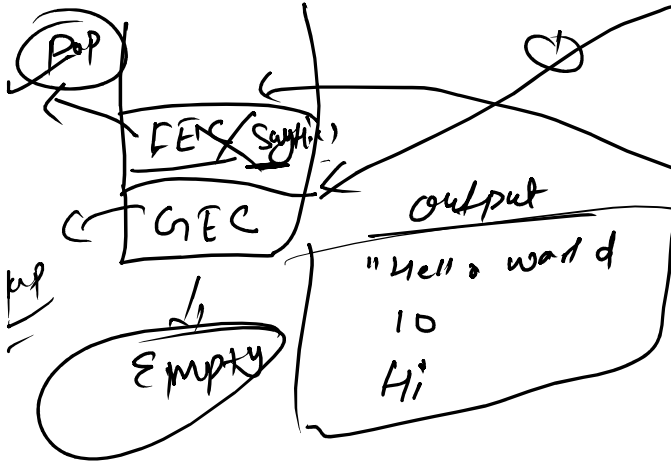
memory

- `a : undefined` with a downward arrow pointing to the value `10`.
- `SayHi : function()`

code

- `console.log("Hello")`
- `a = 10`
- `log(a)` (with a circled 'X' next to it, indicating an error or undefined behavior)
- `SayHi()` (highlighted with a box)

Call Stack



function execution context

memory	code
	<u>console.log('hi');</u>

momary

c d e

Consolo. $\log(^n 4;)$